

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
df = pd.read_csv("news.csv",engine="python",on_bad_lines="skip")
df.head()
```

|   | text                                               | label |
|---|----------------------------------------------------|-------|
| 0 | Gere faults Trump for blurring meaning of 'ref...  | 1     |
| 1 | German parties start to find common ground in ...  | 1     |
| 2 | Senate Democratic leader says Attorney General...  | 1     |
| 3 | Tennis: Kyrgios fined \$10,000 for Shanghai wal... | 1     |
| 4 | Trump Threw Mar-A-Lago Fundraiser For Woman A...   | 0     |

```
df.info()
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 43653 entries, 0 to 43652
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0    text    43653 non-null    object
1    label    43653 non-null    int64
dtypes: int64(1), object(1)
memory usage: 682.2+ KB
```

```
0
text 0
label 0

dtype: int64
```

```
df = df.dropna()
```

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df["label"] = le.fit_transform(df['label'])
```

```
X = df.drop("label",axis=1)
y = df["label"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(stop_words='english')

X_train = vectorizer.fit_transform(X_train)
X_test = vectorizer.transform(X_test)
```

```
print(X_train.shape)
print(y_train.shape)
```

```
(1, 1)
(34922,)
```

```
print(df.columns)
```

```
Index(['text', 'label'], dtype='object')
```

```
print(df.columns)
df.head()
```

```
Index(['text', 'label'], dtype='object')
```

|   | text                                               | label |
|---|----------------------------------------------------|-------|
| 0 | Gere faults Trump for blurring meaning of 'ref...  | 1     |
| 1 | German parties start to find common ground in ...  | 1     |
| 2 | Senate Democratic leader says Attorney General...  | 1     |
| 3 | Tennis: Kyrgios fined \$10,000 for Shanghai wal... | 1     |
| 4 | Trump Threw Mar-A-Lago Fundraiser For Woman A...   | 0     |

```
print(X_train.shape)
print(y_train.shape)
```

```
(1, 1)
(34922,)
```

```
type(X_train)
```

```
scipy.sparse._csr.csr_matrix
def __init__(arg1, shape=None, dtype=None, copy=False, *, maxprint=None)
```

Compressed Sparse Row matrix.

This can be instantiated in several ways:  
 csr\_matrix(D)  
 where D is a 2-D ndarray

```
X = df['text']
y = df['label']
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(stop_words='english')
```

```
X_train = vectorizer.fit_transform(X_train)
X_test = vectorizer.transform(X_test)
```

```
print(X_train.shape)
print(y_train.shape)
```

```
(34922, 139551)
(34922,)
```

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

▼ LogisticRegression ⓘ ?  
 LogisticRegression(max\_iter=1000)

```
y_pred = model.predict(X_test)
```

```
from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Accuracy: 0.9712518611842859

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test, y_pred))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.97      | 0.97   | 0.97     | 4351    |
| 1            | 0.97      | 0.97   | 0.97     | 4380    |
| accuracy     |           |        | 0.97     | 8731    |
| macro avg    | 0.97      | 0.97   | 0.97     | 8731    |
| weighted avg | 0.97      | 0.97   | 0.97     | 8731    |

```
news = ["Scientists discover a new method to generate clean energy"]
```

```
news_vector = vectorizer.transform(news)
```

```
prediction = model.predict(news_vector)
```

```
if prediction[0] == 1:
    print("Fake News")
else:
    print("Real News")
```

Real News

```
import joblib
```

```
joblib.dump(model, "fake_news_model.pkl")
joblib.dump(vectorizer, "vectorizer.pkl")
```

['vectorizer.pkl']

```
from google.colab import files
```

```
files.download("fake_news_model.pkl")
files.download("vectorizer.pkl")
```